LOC Fall 2026 COSI 455

LegacyBTC

A Bitcoin Wallet with Digital Asset Inheritance

Biynah Dayok

Dr. Wei Ding

August 30, 2026

LeMoyne-Owen College

Short Description

LegacyBTC is an Android-based Bitcoin wallet that will allow users to create and manage a Bitcoin wallet while providing a method for transferring digital assets to a designated beneficiary. One major problem with self-custodied Bitcoin is that if an owner dies or permanently loses access to the wallet, the Bitcoin can become inaccessible to family members or other intended beneficiaries. LegacyBTC will address this problem by combining normal Bitcoin wallet functionality with a digital inheritance feature.

The application will allow users to create or restore a Bitcoin wallet, generate Bitcoin receiving addresses, send and receive Bitcoin, view wallet balances, and review transaction history. A wallet owner will also be able to designate a beneficiary and establish an inheritance period. The inheritance system will use Bitcoin's time-based transaction capabilities to create a recovery path that becomes available to the designated beneficiary after the required conditions are satisfied. The project will use Java, Android Studio, and the bitcoinj library to implement Bitcoin wallet and transaction functionality. The application will be developed and tested on a Bitcoin testing network before final mainnet compatibility is implemented.

1. Application Domain

LegacyBTC will operate within the following application domains:

- Blockchain technology
- Bitcoin and cryptocurrency
- Financial technology (FinTech)
- Digital asset management
- Digital inheritance
- Mobile application development

The primary real-world problem addressed by the application is the inheritance of self-custodied Bitcoin. Unlike assets held by traditional financial institutions, Bitcoin controlled through private keys can become permanently inaccessible if the owner dies or loses the required credentials.

LegacyBTC will provide a Bitcoin wallet that allows an owner to establish a beneficiary and an inheritance recovery path for their Bitcoin.

2. Programming Languages, Technologies, and IDE

Java

Java will be the primary programming language for the project. It will be used for:

- Android application logic
- Object-oriented programming
- Wallet management
- Transaction processing
- Input validation
- Inheritance management
- User interactions
- Bitcoin network interactions

XML

XML will be used to design the Android application's user interfaces, including buttons, text fields, menus, lists, and other visual components.

Android Studio

Android Studio will be the primary IDE used to design, develop, debug, test, and package the LegacyBTC Android application.

bitcoinj

The bitcoinj Java library will provide the application's core Bitcoin functionality. It will be used for:

- Bitcoin wallet creation
- Bitcoin wallet recovery
- Cryptographic keys

- Bitcoin addresses
- Bitcoin transactions
- Transaction signing
- Bitcoin network communication

Gradle

Gradle will be used for project configuration and dependency management.

3. Major User Interfaces

LegacyBTC will contain seven major user interfaces.

3.1 Create/Restore Wallet Screen

This will be the first screen displayed when a user begins using LegacyBTC.
The user will be able to:

- Create a new Bitcoin wallet
- Restore an existing Bitcoin wallet
- Enter required recovery information
- Continue to the wallet dashboard

Basic Controls

- Create Wallet button
- Restore Wallet button
- Recovery information input fields
- Continue button
- Back button

3.2 Wallet Dashboard

The dashboard will serve as the main screen of LegacyBTC.

It will display:

- Bitcoin wallet balance
- Bitcoin wallet address
- Recent transactions
- Inheritance status

Basic Controls

- Send button
- Receive button
- Inheritance button
- Transaction History button
- Settings button

3.3 Send Bitcoin Screen

This screen will allow the owner to send Bitcoin from the wallet.

Inputs

- Recipient Bitcoin address
- Bitcoin amount

Basic Controls

- Recipient address field
- Amount field
- Send button

- Cancel button
- Transaction confirmation dialog

The application will validate the address, amount, available wallet balance, and other required information before creating the transaction.

3.4 Receive Bitcoin Screen

This screen will allow the user to receive Bitcoin.

The application will display:

- Bitcoin receiving address
- QR code representing the address

Basic Controls

- Copy Address button
- Generate Address button
- Share button
- Back button

Another Bitcoin user will be able to scan the QR code or copy the displayed address to send Bitcoin to the LegacyBTC wallet.

3.5 Inheritance Setup Screen

This screen will allow the Bitcoin owner to configure the digital inheritance feature.

The owner will provide:

- Beneficiary Bitcoin information/public key
- Bitcoin amount covered by the inheritance plan
- Inheritance/timelock period

The application will display:

- Beneficiary information
- Inheritance period
- Current inheritance status
- Bitcoin associated with the inheritance arrangement

Basic Controls

- Add Beneficiary button
- Set Inheritance Period button
- Save Inheritance Plan button
- Update Plan button
- Cancel Plan button

3.6 Beneficiary/Recovery Screen

The beneficiary screen will provide information about an inheritance arrangement.

It will display:

- Inheritance plan status
- Inheritance amount
- Timelock status
- Beneficiary Bitcoin information
- Whether the recovery condition has been satisfied

Once the required inheritance conditions have been satisfied, the beneficiary will be able to initiate the recovery/claim process.

Basic Controls

- Check Status button
- Claim/Recover button
- View Details button

3.7 Transaction History Screen

This screen will display Bitcoin transactions associated with the wallet.

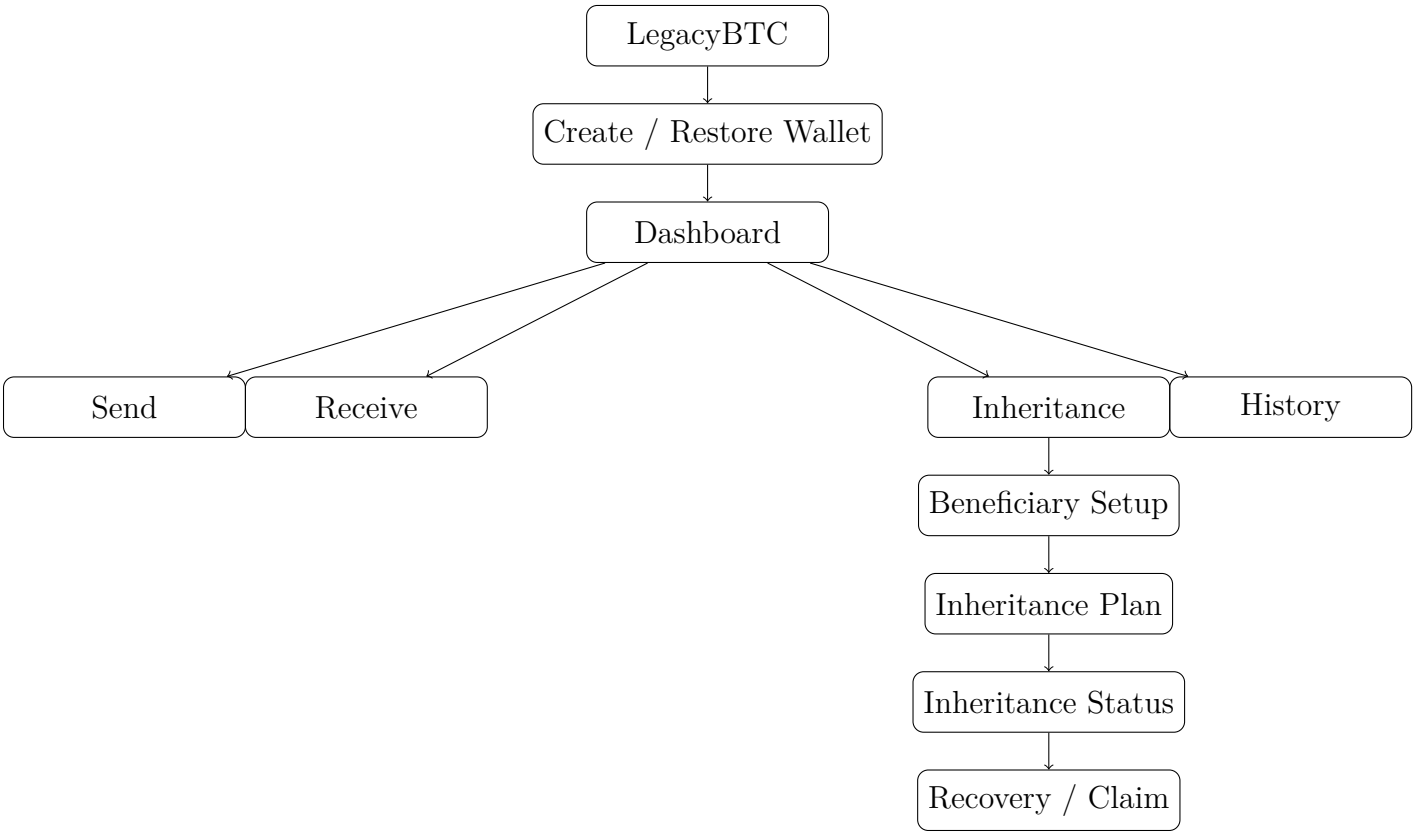
Information will include:

- Transaction type
- Bitcoin amount
- Transaction ID
- Transaction status
- Date/time

The application will use a scrollable list to display transaction records.

4. Basic Interface Design

The major navigation structure of LegacyBTC will be:



The user interface will use standard Android components such as:

- Buttons
- Text fields
- Text labels
- Lists
- Menus
- Confirmation dialogs
- QR codes
- Navigation controls

5. User Interaction

A user will begin by creating a new Bitcoin wallet or restoring an existing wallet.

After accessing the wallet, the user will enter the dashboard and view the wallet balance, Bitcoin address, recent transactions, and inheritance status.

To receive Bitcoin, the user will select **Receive**. LegacyBTC will generate or display a Bitcoin receiving address and QR code.

To send Bitcoin, the user will select **Send**, enter the recipient's Bitcoin address and amount, review the transaction, and confirm it. The application will validate the transaction information before the Bitcoin transaction is created, signed, and submitted to the Bitcoin network.

To establish digital inheritance, the owner will select **Inheritance**, provide the beneficiary information, choose the inheritance period, and save the inheritance plan.

The application will track the inheritance conditions and display the status of the inheritance arrangement. Once the required Bitcoin timelock conditions have been satisfied, the beneficiary recovery path will become available and the beneficiary will be able to initiate the recovery process.

6. Input and Output

Inputs

LegacyBTC will accept the following inputs:

- Bitcoin wallet recovery information
- Bitcoin recipient addresses
- Bitcoin transaction amounts
- Beneficiary Bitcoin public key/address information
- Inheritance period
- User confirmations
- QR-code information
- Bitcoin network information

Outputs

LegacyBTC will display or generate:

- Bitcoin wallet addresses
- Bitcoin wallet balance
- QR codes
- Transaction history
- Transaction confirmations
- Transaction IDs
- Transaction status
- Inheritance plan information
- Beneficiary information
- Inheritance status

- Timelock status
- Recovery status
- Validation messages
- Error messages

7. Primary Algorithms

Wallet Creation and Recovery

The application will use bitcoinj wallet functionality to generate or recover the cryptographic information required to manage a Bitcoin wallet.

Bitcoin Address Generation

The application will generate valid Bitcoin addresses associated with the user's wallet for receiving Bitcoin.

Transaction Processing

The application will construct Bitcoin transactions using information including:

- Recipient Bitcoin address
- Amount
- Available Bitcoin
- Required transaction information

Transaction Signing

Transactions will be cryptographically signed using the appropriate wallet credentials before being submitted to the Bitcoin network.

Input Validation

The application will validate user inputs before performing important operations.

For example:

```
IF recipient address is invalid
    Display error

ELSE IF amount <= 0
    Display error

ELSE IF amount > available balance
    Display insufficient balance

ELSE
    Continue with transaction
```

Inheritance Condition Algorithm

The application will determine whether the inheritance recovery path is currently available based on the inheritance/timelock conditions.

For example:

```
Retrieve inheritance plan

Check inheritance status

Check required timelock condition

IF condition has not been satisfied
    Recovery remains locked

ELSE
    Beneficiary recovery path becomes available
```

8. Primary Data Structures

LegacyBTC will use Java classes and collections to organize application information.

Major classes will include:

```
User
|-- Owner
|-- Beneficiary

BitcoinWallet
|-- Addresses
|-- Transactions

InheritancePlan
|-- Owner
|-- Beneficiary
|-- Bitcoin Amount
|-- Inheritance Period
|-- Status

BitcoinTransaction

BitcoinService
```

Java collections such as `ArrayList` will be used to store and process collections of objects such as Bitcoin transactions.

For example:

```
ArrayList<BitcoinTransaction> transactions;
```

The program will loop through the collection when displaying transaction history or processing transaction information.

9. Required Syntactical Components

LegacyBTC will implement **selection, loops, and functions/methods** throughout the application.

Selection

Selection statements such as `if`, `else if`, and `else` will be used to make decisions.

For example:

```
if (amount > balance) {
    displayInsufficientBalance();
} else {
    createTransaction();
}
```

Selection will also be used for:

- Validating Bitcoin addresses
- Checking wallet balances
- Checking transaction status
- Checking inheritance status
- Determining whether recovery conditions have been satisfied

Loops

Loops will be used to process collections of information.

For example:

```
for (BitcoinTransaction transaction : transactions) {
    displayTransaction(transaction);
}
```

Loops will be used for:

- Processing transaction history
- Searching transaction records
- Processing wallet information
- Displaying collections of application objects

Functions and Methods

Functions and methods will separate major application operations into reusable components.

Methods will include:

```
createWallet()
restoreWallet()
generateAddress()
getBalance()
sendBitcoin()
validateAddress()
getTransactions()
addBeneficiary()
createInheritancePlan()
checkInheritanceStatus()
recoverInheritance()
```

10. Object-Oriented Programming Features

LegacyBTC will implement **encapsulation, inheritance, and polymorphism/dynamic binding** as part of its Java object-oriented design.

10.1 Encapsulation

Encapsulation will protect important application information by keeping object data private and controlling how other parts of the application access or modify that information.

For example, the `InheritancePlan` class will contain private fields:

```
class InheritancePlan {

    private String beneficiaryAddress;
    private long inheritancePeriod;
    private boolean active;

    public String getBeneficiaryAddress() {
        return beneficiaryAddress;
    }

    public void setBeneficiaryAddress(String address) {
        beneficiaryAddress = address;
    }
}
```

Other parts of the application will not directly modify the private inheritance information.

Instead, they will use controlled methods such as:

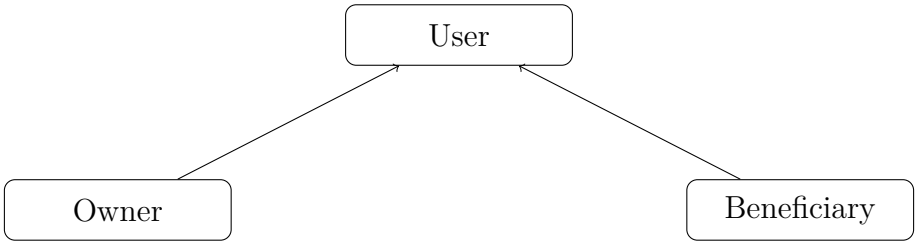
```
getBeneficiaryAddress()
setBeneficiaryAddress()
checkInheritanceStatus()
```

This will demonstrate encapsulation by keeping important data and the methods that manage that data together inside the appropriate classes.

10.2 Inheritance

Inheritance will be implemented using a parent `User` class and specialized `Owner` and `Beneficiary` subclasses.

The class relationship will be:



The `User` class will contain information and functionality shared by both user types.

For example:

```
class User {

    protected String bitcoinAddress;

    public String getBitcoinAddress() {
        return bitcoinAddress;
    }
}
```

The `Owner` class will inherit from `User` and provide owner-specific functionality:

```
class Owner extends User {

    public void createInheritancePlan() {
        // Create inheritance plan
    }
}
```

The `Beneficiary` class will also inherit from `User` and provide beneficiary-specific functionality:

```
class Beneficiary extends User {

    public void checkInheritance() {
        // Check inheritance status
    }
}
```

This will allow common properties and methods to be reused while each subclass provides functionality appropriate to its role.

10.3 Polymorphism / Dynamic Binding

Polymorphism and dynamic binding will be implemented by defining common behavior in the `User` parent class and overriding that behavior in the `Owner` and `Beneficiary` subclasses.

For example:

```
class User {

    public void performWalletAction() {
        System.out.println("Performing wallet action");
    }
}
```

The `Owner` class will override the method:

```
class Owner extends User {

    @Override
    public void performWalletAction() {
        System.out.println(
            "Managing inheritance plan"
        );
    }
}
```

The `Beneficiary` class will provide a different implementation:

```
class Beneficiary extends User {

    @Override
```

```
    public void performWalletAction() {
        System.out.println(
            "Checking inheritance recovery"
        );
    }
}
```

The application will then be able to use:

```
User user = new Owner();
user.performWalletAction();
```

Although the variable is declared as a `User`, Java will determine at runtime that the object is an `Owner` and execute the `Owner` implementation.

This will demonstrate **polymorphism through dynamic method binding**.

Therefore, LegacyBTC will explicitly implement all three OOP features:

- **Encapsulation:** Private wallet and inheritance information will be controlled through class methods.
- **Inheritance:** `Owner` and `Beneficiary` will inherit common properties and methods from `User`.
- **Polymorphism/Dynamic Binding:** `Owner` and `Beneficiary` will override shared methods, and Java will select the appropriate implementation at runtime.

11. Project Timeline

Weeks 1–2: Project Setup and Application Design

- Create the Java Android Studio project
- Configure Gradle and required dependencies
- Integrate bitcoinj
- Design application architecture
- Create primary Java classes
- Design the major user interfaces
- Implement navigation between screens

Weeks 3–4: Bitcoin Wallet Development

- Implement wallet creation
- Implement wallet restoration
- Generate Bitcoin addresses
- Display wallet information
- Implement secure handling of wallet information
- Connect the application to the Bitcoin network

Weeks 5–6: Bitcoin Transaction Functionality

- Implement Bitcoin receiving functionality
- Generate receiving QR codes
- Display Bitcoin balance
- Implement Bitcoin transaction creation
- Implement transaction signing
- Implement sending functionality
- Display transaction history

Weeks 7–8: Digital Inheritance Development

- Implement the `Owner` and `Beneficiary` classes
- Implement the `InheritancePlan` class
- Implement beneficiary configuration
- Implement inheritance/timelock logic
- Implement inheritance status checking
- Implement beneficiary recovery workflow

Weeks 9–10: Integration and Testing

- Integrate wallet and inheritance functionality
- Test wallet creation and restoration
- Test Bitcoin transactions
- Test inheritance conditions
- Test beneficiary recovery
- Improve input validation
- Improve error handling
- Fix application bugs

Weeks 11–12: Finalization

- Complete application testing
- Improve application security
- Finalize user interface
- Complete project documentation
- Prepare project demonstration
- Prepare final project presentation

12. Weekly Meeting with Instructor

Day: Thursday

Time: 1:00 PM